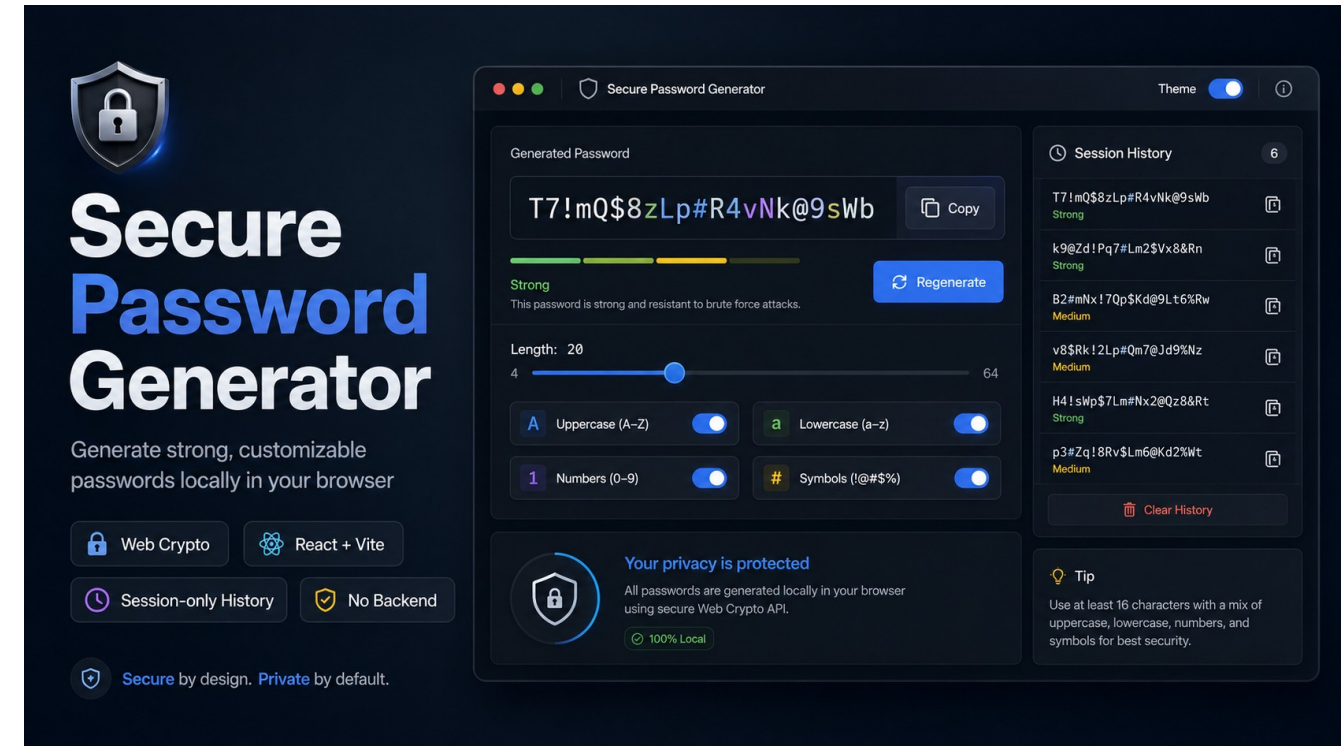


Secure Password Generator

Architecture, Security Logic, React Workflow, Privacy, Testing and Learning Outcomes



React 19 Frontend	Vite 7 Build Tool	Web Crypto Secure Random	No Backend Privacy-first
-----------------------------	-----------------------------	------------------------------------	------------------------------------

Created by

Ann Noor Muhammad Mrinmoy

August 2026 | Portfolio / University Demonstration / Viva

Executive Overview

Secure Password Generator is a privacy-first, frontend-only security utility built with React and Vite. It generates cryptographically secure passwords and passphrases directly inside the browser using the Web Crypto API. The project deliberately avoids a backend, database, account system, external password API, and permanent storage of generated credentials.

The application combines secure randomness, configurable password rules, passphrase generation, strength and entropy estimation, multiple-output generation, session-only history, a local password analyzer, responsive light/dark UI, accessibility-focused controls, and a built-in dependency-free security audit.

Portfolio positioning

This is not just a random-string generator. Its portfolio value comes from showing security-aware frontend engineering: cryptographically secure randomness, modulo-bias avoidance, privacy boundaries, responsible security wording, accessible UI, and testable invariants.

Project Snapshot

Project type	Frontend security utility
Language	JavaScript + JSX + HTML + CSS
Framework	React 19.1.1
Build tool	Vite 7.1.1 + @vitejs/plugin-react
UI icons	Lucide React 1.20.0
Security API	Web Crypto API - <code>crypto.getRandomValues()</code>
Temporary secret storage	sessionStorage - maximum 12 generated credentials
Persistent storage	localStorage only for theme and non-sensitive base settings
Backend / Database	None
Node requirement	Node.js >= 20.19.0

What this guide will help you explain

- Why the project exists and how the idea can be presented in a portfolio.
- How React state, components, hooks and utilities cooperate.
- Why `Math.random()` is not suitable for password generation.
- How rejection sampling removes modulo bias.
- How required character classes are guaranteed and then securely shuffled.
- How entropy and strength estimates are calculated without making misleading claims.
- Why `sessionStorage` and `localStorage` are used for different purposes.
- How the UI handles accessibility, responsiveness, clipboard behavior and privacy.
- How to run, audit, build, customize and explain the project in a viva.

Contents

01. Idea and problem statement

- 02. Goals and design principles
- 03. Technology stack and tools
- 04. Project architecture and folder structure
- 05. React application workflow

06. Secure randomness and Web Crypto

- 07. Password-generation algorithm
- 08. Validation and exclusions
- 09. Passphrase mode
- 10. Strength and entropy system
- 11. Storage, history and privacy
- 12. Clipboard, visibility and keyboard shortcuts
- 13. UI/UX and responsive design
- 14. Accessibility

15. Testing and final audit

- 16. How to run and build
- 17. How to customize the project
- 18. What I learned
- 19. Limitations and future improvements
- 20. Viva / interview quick preparation

WHY THIS PROJECT

1. Project Idea and Problem Statement

The idea in one sentence

The project can be explained as starting from one practical question: "Can I build a password generator that is not only convenient, but also technically responsible, privacy-first, and secure enough to demonstrate real frontend security concepts?"

সাধারণ password generator বানানো খুব সহজ - কিছু character array থেকে random character নিয়ে string বানালেই হয়। কিন্তু security project হিসেবে সেটা যথেষ্ট না। কারণ random selection যদি predictable হয়, enabled category miss করে, secret permanent storage-এ থেকে যায়, বা UI misleading security claim দেয় - তাহলে app দেখতে ভালো হলেও technically weak হয়ে যায়।

এই project-এর design story তাই তিনটা problem-এর উপর দাঁড়ায়: secure randomness, correct password rules, এবং privacy. এর সাথে portfolio value বাড়ানোর জন্য passphrase mode, strength analysis, history, responsive UI, accessibility এবং audit script যোগ করা হয়েছে।

Simple way to explain it in viva

"I wanted to build a frontend-only password utility where the browser itself generates the secret using Web Crypto. My main focus was not only UI; I also handled unbiased random selection, required-character guarantees, temporary history, safe storage boundaries, strength estimates, and accessibility."

Problems the project solves

- Users need long, unique and customizable credentials without sending them to a server.
- Random output must not accidentally omit an enabled category such as numbers or symbols.
- Similar characters like 0/O and 1/I/I may be inconvenient when manually reading a password.
- Some websites restrict allowed symbols, so custom symbol control is useful.
- Users may want readable passphrases instead of dense random passwords.
- A strength meter should educate, not falsely claim that a credential is "unbreakable".
- Generated history is useful, but permanent secret persistence increases exposure.

DESIGN INTENT

2. Goals and Design Principles

Security first	Use browser cryptographic randomness for every security-sensitive selection and shuffle.
Local by design	Password/passphrase generation occurs in the browser with no server round trip.
Correctness	Validate lengths, enabled categories, exclusions and empty pools before generation.
Privacy boundary	Keep generated history session-only and persistent settings non-sensitive.

Responsible UX	Use estimates and disclaimers instead of fake crack-time precision or absolute security claims.
Portfolio quality	Polished dark/light UI, mobile responsiveness, components, utility separation, docs and audit script.
Accessibility	Labels, switch semantics, keyboard support, focus states, live feedback and reduced-motion handling.

WHAT WAS USED

3. Technology Stack and Tools

Runtime technologies

React 19.1.1	Builds the interactive UI using components, state and hooks.
React DOM 19.1.1	Renders the React application into the browser DOM.
JavaScript / JSX	Implements application logic, security utilities and component markup.
Vite 7.1.1	Development server, module bundling and production build.
Modern CSS	Layout, theme variables, responsive breakpoints, focus states and visual polish.
Lucide React	Consistent icon set for copy, shield, theme, history, visibility and other controls.
Web Crypto API	Provides cryptographically secure random values through <code>crypto.getRandomValues()</code> .
Browser Storage APIs	<code>sessionStorage</code> for generated history; <code>localStorage</code> only for approved preferences.
Clipboard API	<code>navigator.clipboard.writeText()</code> for copy actions with a fallback path.

Development workflow tools

The source project is configured for a standard Node.js + npm workflow. In practice, a code editor such as VS Code and Git/GitHub can be used for development and version control, but they are not runtime dependencies of the application.

Core npm commands

```
npm install
npm run dev
npm run audit
npm run build
npm run preview
```

Why React?

React is useful here because the app has many connected UI states: password settings, passphrase settings, theme, generated output, visibility, batch results, toast feedback, advanced options and history. React makes these state transitions easier to keep predictable and componentized.

Bangla explanation: একাধিক setting change হলে UI-এর বিভিন্ন জায়গা একসাথে update করতে হয়। React state ব্যবহার করলে DOM manually manipulate না করে state পরিবর্তন করলেই relevant component re-render হয়।

Why Vite?

Vite gives a fast development server and a simple optimized production build. The project uses a minimal Vite configuration with the React plugin and host access enabled for development testing on the local network.

HOW THE CODE IS ORGANIZED

4. Project Architecture and Folder Structure

Frontend-only architecture

Security-sensitive operations remain inside the browser; no backend or database exists.

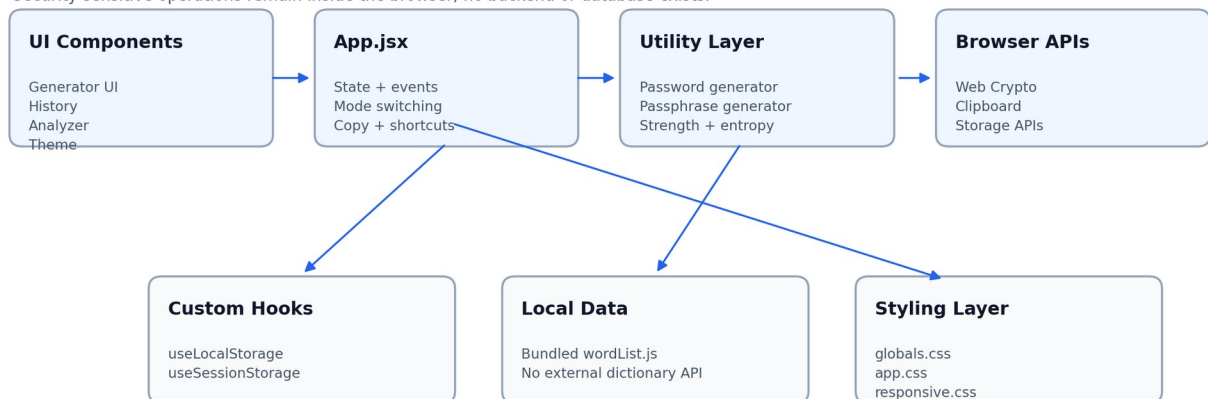


Figure 1 - High-level architecture of the frontend-only application.

Actual audited project structure

```

secure-password-generator/
  scripts/
    security-audit.mjs
  src/
    components/
      PasswordAnalyzer.jsx
      SessionHistory.jsx
      StrengthMeter.jsx
      Toast.jsx
      Toggle.jsx
    data/
      wordList.js
    hooks/
      useLocalStorage.js
  
```

```

    useSessionStorage.js
  utils/
    entropyCalculator.js
    passphraseGenerator.js
    passwordGenerator.js
    secureRandom.js
    storageUtils.js
    strengthCalculator.js
  styles/
    app.css
    globals.css
    responsive.css
  App.jsx
  main.jsx
index.html
package.json
vite.config.js
README.md
PROJECT_GUIDE.md
FINAL_AUDIT_REPORT.md

```

Responsibility of important files

src/App.jsx	Main orchestration: application state, generation workflow, copy actions, mode switching, theme, history integration, batch generation and keyboard shortcuts.
src/utils/secureRandom.js	Central security randomness layer: getRandomValues(), unbiased random index, secure choice and secure Fisher-Yates shuffle.
src/utils/passwordGenerator.js	Character pools, settings normalization, exclusions, validation, required-category guarantee and final password generation.
src/utils/passphraseGenerator.js	Validates passphrase settings and selects bundled words / optional two-digit number securely.
src/utils/entropyCalculator.js	Safe entropy calculations and broad entropy/resistance labels.
src/utils/strengthCalculator.js	Heuristic strength score, common-pattern detection, sequence detection, repetition penalty and diversity checks.
src/hooks/useLocalStorage.js	Reusable persistence hook for non-sensitive preferences.
src/hooks/useSessionStorage.js	Reusable session-scoped persistence hook for generated history.
src/utils/storageUtils.js	Safe JSON parsing plus guarded browser storage reads/writes.
src/components/SessionHistory.jsx	Reveal/hide, copy, reuse settings, delete and clear temporary history.
src/components/PasswordAnalyzer.jsx	Analyzes a typed password in component state only; does not persist analyzer input.

`src/styles/*.css`

Global theme, layout, component styling and responsive behavior.

STATE AND INTERACTION

5. React Application Workflow

App.jsx is the coordination layer. It owns the main state and passes focused data/actions to reusable components. This avoids scattering security logic across visual components.

Main state groups

- Theme: persisted as light/dark preference.
- Password settings: length, uppercase, lowercase, numbers, symbols, exclude-similar, advanced custom exclusions and custom symbols.
- Passphrase settings: word count, separator, capitalization and optional random number.
- Output state: current generated credential, computed metrics and visibility.
- Batch state: 1/3/5/10 generated values.
- History state: normalized sessionStorage entries.
- UX state: toast messages and advanced-options expansion.

Generation event flow

1. User changes settings. Base non-sensitive password preferences are synchronized to localStorage.
2. User presses Generate (or shortcut G outside editable fields).
3. The current mode is validated: password or passphrase.
4. The secure generator returns a new credential.
5. Metrics are calculated using the generation-time configuration.
6. The current output and metrics are updated together.
7. The credential is added to sessionStorage history with length, strength label, time and reusable generation settings.
8. A non-blocking toast confirms the action.

Important final-audit fix

Metrics are tied to the configuration that generated the current credential. Changing pending settings no longer silently changes the displayed entropy/strength for an older output.

CORE SECURITY LOGIC

6. Secure Randomness and Web Crypto

This is the most important technical part of the project. Security-sensitive randomness is centralized in `src/utls/secureRandom.js`. The application never uses `Math.random()` for password characters, passphrase words, random numbers, or shuffling.

Why Math.random() is avoided

Math.random() is designed for general-purpose pseudo-randomness, not cryptographic secret generation. A password generator needs a cryptographically secure random source whose outputs are much harder to predict from previous values or internal state.

এখানে main lesson হলো: UI random দেখালেই security random হয় না। Password-এর মতো secret generate করার সময় randomness-এর source-টাই security property-এর অংশ।

How crypto.getRandomValues() is used

Simplified secure random source

```
const values = new Uint32Array(1);
cryptoApi.getRandomValues(values);
```

The browser fills a Uint32Array with cryptographically secure random values. The project wraps this inside a reusable helper instead of directly repeating random code throughout the application.

Modulo bias and rejection sampling

Suppose a random 32-bit value is converted to an index with value % N. If 2^{32} is not exactly divisible by N, a small incomplete tail exists, which means some indexes can occur slightly more often than others. For a high-quality security utility, the project avoids that bias.

Actual rejection-sampling idea from secureRandomIndex()

```
const limit = UINT32_RANGE - (UINT32_RANGE % maxExclusive);

do {
  cryptoApi.getRandomValues(values);
} while (values[0] >= limit);

return values[0] % maxExclusive;
```

Bangla explanation: 32-bit range-এর যে অংশ N দিয়ে সমানভাবে ভাগ হয় না, সেই tail-এর random value reject করা হয়। তারপর modulo apply করা হয়। ফলে valid index-গুলোর distribution যতটা সম্ভব uniform থাকে।

Edge-case hardening

secureRandomIndex() explicitly rejects unsupported maxExclusive values above 2^{32} . This prevents a theoretical endless rejection loop for invalid oversized ranges.

Secure Fisher-Yates shuffle

Secure shuffle logic

```
for (let index = output.length - 1; index > 0; index -= 1) {
  const swapIndex = secureRandomIndex(index + 1);
  [output[index], output[swapIndex]] = [output[swapIndex], output[index]];
}
```

The generator first guarantees required categories, then securely shuffles the output. This prevents the guaranteed uppercase/lowercase/number/symbol characters from always appearing in predictable positions.

FROM SETTINGS TO OUTPUT

7. Password-Generation Algorithm

Password generation flow

Each step protects a different invariant: valid input, allowed characters, category guarantees, unbiased randomness, and temporary storage.



Figure 2 - Password generation pipeline implemented by the audited source.

Centralized character pools

Central CHARACTER_POOLS concept

```

uppercase: 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
lowercase: 'abcdefghijklmnopqrstuvwxyz'
numbers:   '0123456789'
symbols:   '!@#$%^&*()_+~[]{};:,.?'
```

Centralizing pools avoids duplicated strings across components. All exclusions and category rules are applied in one utility layer, which makes testing and maintenance simpler.

Required-character guarantee

For every enabled category, generatePassword() securely chooses one character before filling the rest of the password. Therefore, if Uppercase + Lowercase + Number + Symbol are enabled and the length is valid, the final password contains at least one of each.

Core generation logic

```

const output = pools.map((pool) => secureChoice(pool.value));
while (output.length < normalized.length) {
  output.push(secureChoice(combinedPool));
}
return secureShuffle(output).join('');
```

এখানে আগে required character নেওয়া হয়, তারপর remaining position full allowed pool থেকে fill হয়, শেষে secure shuffle হয়। শুধু combined pool থেকে random character নিলে enabled category accidentally বাদ পড়তে পারত।

PREVENTING INVALID OUTPUT

8. Validation, Exclusions and Edge Cases

Main validation rules

- At least one character category must be enabled.
- Password length must be an integer from 4 to 64.
- Length cannot be smaller than the number of enabled required categories.
- An enabled category cannot become empty after exclusions.
- If custom symbols are enabled, the symbol set cannot be empty.
- Custom symbols are restricted to printable ASCII symbols - no letters, digits, spaces or emoji.
- An all-excluded configuration fails with a useful error instead of returning an empty password.

Similar-character exclusion

When enabled, the generator removes 0, O, o, 1, l, I and | before secure selection. This feature is mainly for readability when a user must manually type or communicate a credential.

Custom exclusions

A user may provide characters that should never appear. The exclusion set is applied before generation to each enabled pool. If an exclusion empties a required pool, validation stops generation instead of silently weakening the rules.

Why this matters

Validation is part of security correctness. A generator that silently ignores an enabled category or returns an empty/short credential is functionally incorrect even if its random source is secure.

READABLE SECURE CREDENTIALS

9. Passphrase Mode

Passphrase mode is an alternative to character-dense passwords. It selects 3 to 8 words from a bundled local word list using the same secure-choice helper. No external dictionary service or network request is required.

Word count	3-8
Separator	0-3 characters
Capitalize words	Deterministic formatting option; does not add entropy
Add number	Securely generates a 00-99 value with <code>secureRandomIndex(100)</code>
Word source	<code>src/data/wordList.js</code> - bundled locally
Randomness	Web Crypto only

Passphrase generation concept

```
for (let index = 0; index < settings.wordCount; index += 1) {
  let word = secureChoice(WORD_LIST);
  if (settings.capitalizeWords) word = word.charAt(0).toUpperCase() + word.slice(1);
  words.push(word);
}
```

```
}
if (settings.addNumber) words.push(String(secureRandomIndex(100)).padStart(2, '0'));
```

Important: capitalization and separator are user-chosen deterministic formatting. The final audit correctly avoids counting them as extra entropy. Only independently random choices contribute entropy.

SECURITY ESTIMATES

10. Strength Meter and Entropy System

Password entropy approximation

Formula

$\text{Entropy} \approx \text{Length} \times \log_2(\text{Character Pool Size})$

For generated random passwords, the app estimates entropy using the selected pool size and password length. The helper first normalizes invalid numeric input so the UI cannot show NaN, Infinity or negative entropy.

Example: যদি pool size 72 হয় এবং password length 16 হয়, approximate entropy = $16 \times \log_2(72)$. এটা mathematical estimate; real-world security guarantee নো।

Strength heuristic

The strength score starts from entropy thresholds and then considers practical warning signals. The internal score is clamped from 0 to 4 and mapped to Very Weak, Weak, Fair, Strong or Very Strong.

- Entropy threshold
- Detected character categories
- Common weak patterns such as password, 123456, qwerty, admin and letmein
- Obvious alphabetic/numeric sequences
- Three-or-more repeated characters and heavy repetition
- Short single-category passwords

Responsible wording

The UI deliberately avoids claims such as "unbreakable", "hack-proof" or "100% secure". It uses broad resistance labels - Low, Moderate, High and Very High - plus a visible disclaimer that actual cracking time depends on attack method, hashing algorithm, hardware, leaks, reuse and randomness.

Why the analyzer is different

For a human-typed password, pool-based entropy can overestimate security because humans choose patterns. The analyzer therefore displays the entropy as a rough estimate and also checks patterns, sequences and repetition.

SECRET LIFECYCLE

11. Storage, History and Privacy

Privacy and storage boundary

The project deliberately separates secrets from non-sensitive preferences.

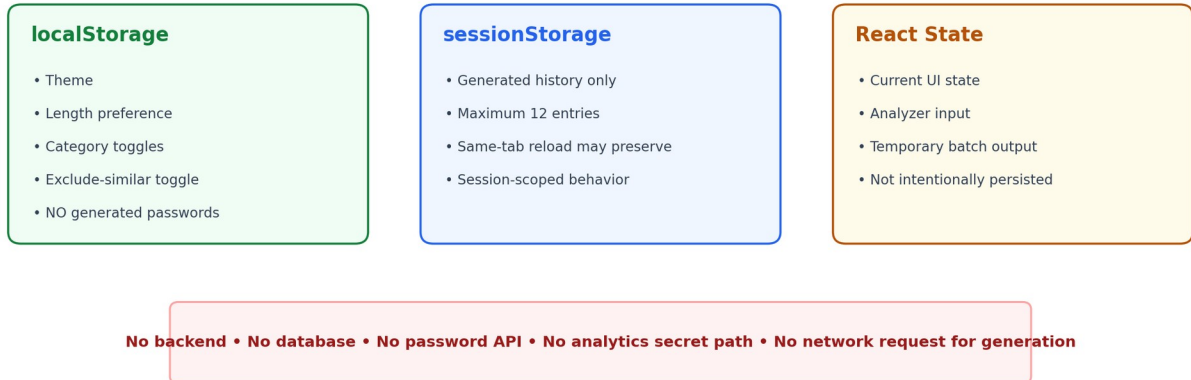


Figure 3 - Separation between persistent preferences, session-only secrets and in-memory analyzer state.

sessionStorage history

Generated credentials are stored under `passwordGenerator.sessionHistory` using `sessionStorage`, limited to the latest 12 entries. Each history item stores the credential, length, strength label, generation time and generation settings so the user can reuse the configuration.

`sessionStorage` সাধারণত একই tab reload করলে থাকে, কিন্তু normal page session শেষ হলে remove হয়। Browser session-restore behavior vary করতে পারে, তাই project documentation absolute guarantee না দিয়ে এই lifecycle accurately explain করে।

localStorage preferences

- `passwordGenerator.theme` - light/dark theme.
- `passwordGenerator.settings` - only base non-sensitive settings: length, category toggles and exclude-similar.
- Generated credential values are not intentionally saved to `localStorage`.
- Advanced custom exclusion text and custom symbol text are intentionally not persisted permanently.

Safe storage parsing

`storageUtils.js` wraps JSON parsing and browser storage reads/writes in `try/catch`. Corrupted JSON, unavailable storage or browser restrictions fall back safely instead of crashing the app.

Password analyzer privacy

Analyzer input stays only in `PasswordAnalyzer` component state. It is not added to generated history, `localStorage`, `sessionStorage`, console logs or network requests.

Visible privacy UI

The application does not hide privacy details only in README. The interface itself states local generation, no account/database, session-only history and clipboard limitations.

12. Clipboard, Visibility and Keyboard Shortcuts

Copy system

The primary path uses `navigator.clipboard.writeText(value)`. A hidden textarea + `document.execCommand("copy")` fallback is included for environments where the modern Clipboard API is unavailable. Copying never regenerates or changes the credential.

The UI warns that a copied credential may remain in the operating-system clipboard until replaced. Browser JavaScript does not promise secure clipboard erasure.

Show / hide behavior

Visibility only changes what is rendered. The underlying generated value remains unchanged, so the copied credential is identical whether it is visually hidden or revealed.

Keyboard shortcuts

G	Generate the current password/passphrase.
C	Copy the current credential.
Safety rule	Shortcuts are ignored while focus is inside input, textarea, select, content-editable fields, or while Ctrl/Alt/Meta is pressed.

PORTFOLIO PRESENTATION

13. UI/UX and Responsive Design

The visual direction is a modern privacy/security utility rather than a hacker-themed interface. The final UI uses restrained blue/security accents, card-based hierarchy, readable typography, dark/light themes, clear generate/copy actions and visible privacy messaging.

Key UI areas

- Header with branding, history shortcut and theme toggle.
- Hero statement emphasizing local/private generation.
- Password/Passphrase mode switch.
- Generated credential output with show/hide, copy and regenerate.
- Length control, presets and category toggles.
- Advanced exclusions and custom symbols.
- Strength/entropy/resistance card.
- Multiple-output generator with Copy All.
- Session history with reveal, copy, reuse settings, delete and clear.
- Local password analyzer, privacy notes and security tips.

Responsive targets

1440px	Balanced desktop workspace with generator and security insight areas.
---------------	---

1024px	Reduced spacing while preserving a desktop/tablet structure.
768px	Panels begin stacking and controls reflow.
430px	Phone-first stacked layout, larger touch targets and wrapped action areas.
375px	Narrow mobile layout with no page-level horizontal overflow.

64-character password handling

Long credentials are not visually destroyed or silently truncated. The output area allows internal horizontal access while the actual copied value remains complete. This is an important UX detail for security tools because a 64-character secret is normal, not an edge case to hide.

INCLUSIVE INTERACTION

14. Accessibility Audit

- Semantic headings and labeled controls.
- Checkboxes exposed with `role="switch"` and `aria-checked`.
- Keyboard-operable native range slider.
- Accessible label for the synchronized number input.
- `aria-label` on icon-only show/hide/copy/delete buttons.
- Strength meter has progressbar semantics and readable text labels.
- Strength is not communicated only by color.
- Toast uses `role="status"` for non-blocking feedback.
- Visible `:focus-visible` states.
- Keyboard shortcuts do not fire while the user is typing.
- Reduced-motion preferences are respected in CSS.

Bangla takeaway: accessibility মানে শুধু screen reader না। Keyboard navigation, visible focus, readable contrast, touch target, text label, motion preference - সবকিছু usable experience-এর অংশ।

HOW CORRECTNESS WAS CHECKED

15. Testing and Final Security Audit

The final project includes `scripts/security-audit.mjs` and an `npm` script: `npm run audit`. The audit is dependency-free and focuses on generator invariants plus source-level security rules.

Automated checks covered

- No `Math.random()` calls in application source.
- No `debug console.log` / `console.debug` / `console.info` paths.
- Repeated generation for lengths 4, 8, 12, 16, 20, 32 and 64.
- Required enabled categories are present.
- Similar-character and custom exclusions work.
- Aggressive exclusions fail safely instead of returning empty output.

- 1, 3, 5 and 10 multiple-generation counts.
- Passphrase behavior for multiple word counts and options.
- Entropy stays finite and non-negative.
- Strength score stays inside the supported range.
- Storage boundaries are checked.

Final-audit fixes worth mentioning

- Passphrase deterministic capitalization no longer falsely adds entropy.
- `secureRandomIndex` rejects impossible ranges above 2^{32} .
- Metrics remain associated with generation-time settings.
- Corrupted/wrong-shaped browser storage no longer crashes state initialization.
- Custom symbol mode rejects letters, numbers, spaces and emoji.
- Advanced custom text is no longer persistently saved in `localStorage`.

Build verification note

In the final audit environment, `npm install` was attempted but the sandbox could not resolve `registry.npmjs.org` (EAI_AGAIN). Because dependencies could not be downloaded there, the Vite production build could not be completed in that environment. Dependency-independent generator audits, source scans, JS/JSX syntax checks and CSS parsing passed. On a normal machine with `npm` registry access, run the commands in the next section.

LOCAL WORKFLOW

16. How to Run, Audit and Build

Requirements

- Node.js 20.19.0 or newer (as declared in `package.json`).
- `npm` with internet access for the first dependency installation.
- A modern browser with Web Crypto support.

Development

Windows example

```
cd F:\Projects\secure-password-generator
npm install
npm run dev
```

Vite will print a local URL such as `http://localhost:5173`. Open it in the browser and test password generation, copy, history, themes and responsive layout.

Security audit

```
npm run audit
```

Production build

```
npm run build
npm run preview
```


npm run build creates the optimized dist/ output. npm run preview serves that production build locally so you can verify it before deployment.

WHERE TO EDIT

17. How to Customize the Project

Default password length	Edit DEFAULT_SETTINGS.length inside App.jsx.
Character pools	Edit CHARACTER_POOLS in src/utils/passwordGenerator.js.
Similar characters	Edit SIMILAR_CHARACTERS in passwordGenerator.js.
Default symbols	Edit CHARACTER_POOLS.symbols.
Passphrase words	Edit src/data/wordList.js; keep it local and curated.
Entropy thresholds	Edit entropyLabel() / resistanceLabel() in entropyCalculator.js.
Strength heuristic	Edit COMMON_PATTERNS, sequences or penalties in strengthCalculator.js.
History limit	Current normalization and addHistory logic use a 12-entry limit.
Theme / colors	Edit CSS variables and theme rules in globals.css / app.css.
Responsive breakpoints	Edit responsive.css.
Icons	Replace Lucide icons at component level without changing core logic.

Safe customization rule

If you change security logic, rerun npm run audit and manually test edge cases. Visual changes are low-risk; randomness, pool, validation and storage changes require stronger review.

LEARNING OUTCOMES

18. What I Learned from This Project

এই project-এর সবচেয়ে valuable অংশ হলো security concept এবং frontend engineering একসাথে practice করা। শুধু React UI বানানো না, বরং security-sensitive decision কেন different হতে হয় সেটা বোঝা যায়।

Cryptographic randomness

Math.random() এবং Web Crypto-এর difference; secure secret generation-এর জন্য CSPRNG কেন দরকার.

Modulo bias

Simple modulo mapping কেন slightly biased হতে পারে এবং rejection sampling দিয়ে সেটা কীভাবে avoid করা যায়.

Secure shuffle

Fisher-Yates algorithm এবং secure random index use করে predictable placement কীভাবে remove করা হয়.

Validation invariants

Enabled category guarantee, length rule, empty pool এবং exclusion edge case কীভাবে আগে থেকেই validate করতে হয়.

React architecture

State, useEffect, useCallback, custom hooks এবং component separation দিয়ে large interactive UI manage করা.

Browser storage security

sessionStorage এবং localStorage একই জিনিস না; secret lifecycle অনুযায়ী storage choose করতে হয়.

Privacy engineering

No backend রাখলেও clipboard, browser storage এবং UI claims-এর privacy implications থাকে.

Strength estimation

Entropy estimate এবং human-password heuristic-এর limitation বুঝে responsible language use করা.

Accessibility

aria labels, switch semantics, keyboard safety, focus states, progress text এবং reduced-motion support.

Testing mindset

Security feature শুধু implement করলেই হয় না; repeated invariant tests, source scans এবং edge cases দরকার.

Documentation

README, project guide এবং audit report actual implementation-এর সাথে sync রাখা professional engineering-এর অংশ.

RESPONSIBLE ENGINEERING

19. Limitations and Future Improvements

Current limitations

- Strength and entropy are estimates, not guarantees.
- A browser app cannot know the target website hashing algorithm, rate limits or attacker hardware.
- Clipboard contents may remain available to other local applications until replaced.
- sessionStorage is temporary by design but browser session-restore behavior can vary.

- The local passphrase word list is intentionally compact; larger curated lists could improve passphrase entropy.
- The app is not a password manager and does not securely store long-term credentials.
- It does not perform breach checking or password-manager integration.

Good future improvements

- Expand the local word list and display word-list size transparently.
- Add unit tests with a formal test runner while keeping the dependency-free audit script.
- Add Playwright/Cypress responsive and accessibility regression tests.
- Add PWA/offline packaging while preserving no-network generation.
- Add optional export of non-secret settings only.
- Add a password-policy preset system for common website restrictions.
- Use zxcvbn-like local analysis only if bundle/privacy trade-offs are documented.
- Add automated Lighthouse/accessibility checks in CI.

EXPLAIN CONFIDENTLY

20. Viva / Interview Quick Preparation

60-second project explanation

Suggested answer

"Secure Password Generator is a frontend-only React and Vite application. It generates passwords and passphrases locally using the Web Crypto API. I centralized secure randomness in a utility that uses rejection sampling to avoid modulo bias and a secure Fisher-Yates shuffle. Password generation guarantees one character from every enabled category, validates exclusions and empty pools, and then calculates approximate entropy plus a heuristic strength score. Generated history is limited to sessionStorage, while localStorage contains only non-sensitive preferences. I also implemented multiple generation, local password analysis, copy fallback, responsive light/dark UI, accessibility controls, and a built-in security audit."

1. What is the main purpose of this project?

To generate customizable, cryptographically secure passwords/passphrases locally in the browser while demonstrating privacy-aware frontend security engineering.

2. Why is there no backend?

Generation does not require one. Keeping secrets local removes unnecessary server transmission and database exposure.

3. Why not use Math.random()?

It is not a cryptographically secure random source and is inappropriate for generating secrets.

4. What does crypto.getRandomValues() do?

It fills typed arrays with cryptographically secure random values provided by the browser.

5. What is modulo bias?

When a random integer range is not evenly divisible by the target range, direct modulo can make some indexes slightly more likely.

6. How do you avoid modulo bias?

By rejection sampling: reject values from the incomplete tail, then apply modulo only to the evenly divisible range.

7. How do you guarantee every enabled character class?

Securely pick one character from each enabled pool first, fill remaining slots from the combined pool, then securely shuffle.

8. Why shuffle after guaranteeing classes?

Without shuffling, required characters could stay in predictable positions.

9. What shuffle algorithm is used?

Fisher-Yates style swapping with secure random indexes.

10. What happens if all categories are disabled?

Validation blocks generation and returns a clear error.

11. What happens if exclusions empty a pool?

Generation stops with a validation message instead of producing invalid output.

12. What is password entropy?

A mathematical estimate of uncertainty; here it is approximated as $\text{length} \times \log_2(\text{pool size})$ for generated random passwords.

13. Is the entropy display exact security?

No. It is an estimate and the UI explicitly presents it as such.

14. How is strength different from entropy?

Strength is a heuristic that starts from entropy but also checks patterns, sequences, repetition, length and character diversity.

15. Why is passphrase capitalization not counted as entropy?

Because capitalization is deterministic when the user enables it; it is not an independent random choice.

16. Why use sessionStorage for history?

It provides temporary same-session convenience without intentionally making generated credentials long-lived persistent data.

17. What is stored in localStorage?

Only theme and non-sensitive base settings such as length, categories and exclude-similar.

18. Is analyzer input stored?

No. It stays in React component state and is not added to history or browser storage.

19. How does copy work?

Primarily `navigator.clipboard.writeText()`, with a small legacy textarea fallback.

20. Can the app securely erase the OS clipboard?

No, so it does not make that claim; it warns that copied credentials may remain until replaced.

21. What does useSessionStorage do?

It initializes React state from sessionStorage and writes state changes back using safe storage helpers.

22. Why use custom hooks?

They remove repeated storage synchronization logic from App.jsx and keep responsibilities cleaner.

23. What accessibility features are important here?

Labels, switch semantics, keyboard slider, focus states, icon-button aria labels, text strength labels, progress semantics and shortcut safety.

24. What is npm run audit?

A dependency-free script that checks generator invariants and source-level security conditions such as no `Math.random()`.

25. What is the strongest learning outcome?

Understanding that secure frontend engineering is a combination of correct algorithms, storage/privacy boundaries, validation, responsible UX and testing - not only visual design.

PORTFOLIO TAKEAWAY

Final Summary

Secure Password Generator is a compact project with unusually strong technical depth for a frontend utility. The visible product is a polished password tool, but the engineering story is broader: cryptographically secure randomness, unbiased selection, secure shuffling, validation invariants, privacy-aware browser storage, heuristic security analysis, accessible interaction, responsive design and repeatable auditing.

Portfolio-তে এই project দেখানোর সময় শুধু "React password generator" বললে project-এর value কমে যাবে। বরং Web Crypto, rejection sampling, session-only history, no backend, analyzer privacy, secure shuffle এবং responsible security estimates - এই differentiatorsগুলো highlight করা উচিত।

One-line portfolio description

A privacy-first React password and passphrase generator using Web Crypto, unbiased secure random selection, session-only history, local strength analysis, responsive UI and accessibility-focused controls.

Recommended repository checks before publishing

1. Run `npm install`.
2. Run `npm run audit` and confirm it passes.
3. Run `npm run build`.
4. Run `npm run preview` and check desktop + mobile widths.
5. Confirm `.gitignore` excludes `node_modules` and build artifacts as intended.
6. Keep `README.md`, `PROJECT_GUIDE.md` and `FINAL_AUDIT_REPORT.md` in the repository.
7. Add the project thumbnail and a few real UI screenshots to the GitHub README/portfolio if desired.